

pamela fox's blog

Tuesday, September 20, 2022

How I setup a Python project

As I prepare for my new role at Microsoft Cloud Advocacy for Python, I've been spinning up a bunch of Python projects over the last few months and learning about tools to improve my development workflow. I'm sharing my project setup process in this post to help other developers start new Python projects and to find out how my process can improve.

My setup process always takes place in a Mac OS X environment, but much of this should be the same across operating systems.

Setup IDE

I use `vim` when editing single files, but when working on multi-file projects I much prefer [VS Code](#). It's fast, it's got great built-in capabilities, and the extensions are incredibly helpful for domain specific tasks.

If it's my first time on a machine, then I need to:

- Download [VS Code](#)
- Download [the Python extension](#)
- Possibly download [Python](#) if my machine's version is outdated

Create folder

Now I make a folder for my project, either via the command line:

```
mkdir my-python-project
```

Or in VS Code, by selecting *File > Open Folder ...* and clicking *New Folder* in the window that pops up.

Initialize version control

I always use [Git](#) for version control in my projects for local source code history, and then use [Github](#) to backup the code to the cloud.

First I create the local Git repository:

```
git init
```

Then I create a [new Github repo](#) and add it as a remote for my local repo:

```
git remote add origin https://github.com/pamelafox/my-python-project-sept7.git
```

Finally I create a `.gitignore` file to tell Git what files/folders to exclude from source control. I'll start it off with a single line to ignore any `__pycache__` folders created automatically by the interpreter.

```
__pycache__/
```

You can learn more on Git/Github from [this Github tutorial](#).

Setup virtual environment

A virtual environment makes it easy to install the dependencies of one Python project without worrying about conflict with dependencies of other projects. It is possible to develop in Python outside of a virtual environment (in the global environment of your system), but that has bitten me so many times that I now always jump into a virtual environment.

Python 3 has built-in support for virtual environments using the [venv module](#). I run this command to create a virtual environment:

[Subscribe to this blog.](#)

ABOUT ME

Currently a Cloud Advocate in Python at Microsoft. Check out my [personal homepage](#) to find out about my projects, talks, code, reading list, and more.

BLOG ARCHIVE

- ▶ [2025](#) (14)
- ▶ [2024](#) (23)
- ▶ [2023](#) (33)
- ▼ [2022](#) (21)
 - ▶ [November](#) (3)
 - ▶ [October](#) (2)
 - ▼ [September](#) (4)
 - [Deploying a containerized Flask app to Azure Conta...](#)
 - [Returning an image from an Azure Function App in P...](#)
 - [Preparing a Django app for deployment on Azure App...](#)
 - [How I setup a Python project](#)
- ▶ [August](#) (4)
- ▶ [July](#) (5)
- ▶ [June](#) (1)
- ▶ [May](#) (1)
- ▶ [January](#) (1)
- ▶ [2020](#) (4)
- ▶ [2018](#) (6)
- ▶ [2017](#) (11)
- ▶ [2016](#) (1)
- ▶ [2015](#) (8)
- ▶ [2014](#) (4)
- ▶ [2013](#) (43)
- ▶ [2012](#) (39)
- ▶ [2011](#) (36)
- ▶ [2010](#) (36)
- ▶ [2009](#) (31)
- ▶ [2008](#) (11)
- ▶ [2007](#) (4)

```
python3 -m venv .venv
```

That creates the environment in the `.venv` folder, so that's where all the project dependencies will get installed. You can name that folder something else, but `.venv` or `venv` are the most standard.

Next I need to actually start the virtual environment, to make sure my terminal is issuing all commands from inside it:

```
source .venv/bin/activate
```

I don't want to check the project's third-party dependencies into my own Git repo, however, both because that's a lot of files to track and because that code should stay in the third-party repos, so I add another line to the `.gitignore` file:

```
.venv/
```

Install requirements

I always install project dependencies with `pip`, which downloads them from the Python package index at pypi.org. However, it's best practice to declare dependencies in a `requirements.txt` file and install from that file, so that anyone else setting up the project will end up with the same dependencies.

I create the `requirements.txt` file and seed it with any packages I know I need:

```
numpy
opencv-python
scikit-image
matplotlib
```

It's also possible to specify the version of each package, but I typically use the latest stable version.

Once that file exists, I ask `pip` to install everything in it:

```
pip install -r requirements.txt
```

Coding time!

This might be the point where I start writing code, or I may wait until I've done the full setup.

Regardless of when I start code writing, this is a typical folder layout for a non-web Python project:

- [main.py](#)
- [image_helpers.py](#)
- [image_filters.py](#)
- tests/
 - [image_filters_test.py](#)
 - [image_helpers_test.py](#)

I have one main module, the entry point, which imports functions or classes from helper modules. Then I put all my tests in a folder, with one test file per module.

At this point, I should be able to actually run the code I've written:

```
python main.py
```

I don't have to worry about distinguishing between the `python` command vs. `python3` command anymore, as I'm running that command inside my virtual environment, and it will always use the version of Python that initialized it.

Install development requirements

Our code might work, but is it *good* code?

Fortunately, there are many tools in the Python ecosystem to help us write better code with a consistent code style.

My favorite tools are all third-party packages, so they need to be installed before I can use them. A best practice is to list them in a separate file, like `requirements-dev.txt`, to make a distinction between the packages needed for production and the additional packages needed for local development.

I create `requirements-dev.txt` and list the tool dependencies:

```
-r requirements.txt
black==22.3.0
pytest==7.1.2
coverage==6.4.1
pytest-cov==3.0.0
pre-commit
ruff
```

*I only include version numbers for the tools here so that you can see how that's specified.

The first line tells pip to also install everything in `requirements.txt`. By including that line, I can now run this single command to install both the prod and dev dependencies:

```
pip install -r requirements-dev.txt
```

Tool configuration

The Python community recently decided on a best practice for tool configuration: putting all options for all tools inside a single file, `pyproject.toml`, so I now create an empty `pyproject.toml` in my root folder.

Run linter

A linter is a tool that looks for "lint" in your code: common coding errors or style issues. The most commonly used Python linter is [flake8](#) but there's a new faster linter available called [ruff](#). It's possible to just use ruff out of the box with its default configuration, but many developers (including myself) like to customize it.

I configure ruff by adding this section to `pyproject.toml`:

```
[tool.ruff]
line-length = 100
ignore = ["D203"]
show-source = true
```

- The `ignore` option tells it what error codes to ignore (that particular one is just a whitespace style nit).
- The `line-length` option specifies the maximum number of characters for each line (a very personal decision!).
- The `show-source` option makes it show the code responsible for any errors.

Now I can run ruff. I actually run it twice, for different levels of errors, first checking for the most serious issues:

```
ruff . --select=E9,F63,F7,F82
```

The error code E9 comes from the [pycodestyle library error codes](#) and refers to any parse errors (IndentationError/SyntaxError) or detectable IO errors.

The other error codes are from the [pyflakes library](#): F63 checks for common issues with comparison operators, F7 covers misplaced statements, and F82 looks for undefined names.

Now I run it again, this time checking for all issues, where the rest is more likely to be style issues. The style violations are based on the [official PEP 8 style guide](#), so they are fairly agreed upon in the Python community.

```
ruff .
```

If I see style issues at this point, I might fix them or I might wait until I introduce the next tool which can autofix many style issues for me. 🤖

Run formatter

A formatter is a tool that will reformat code according to its own standards and your configured options. The most popular formatter is [black](#), which is PEP 8 compliant and highly opinionated.

I configure black by adding this section to `pyproject.toml`:

```
[tool.black]
line-length = 100
target-version = ['py39']
```

Those options tell `black` that I'd like code lines to be no longer than 100 characters and that I'm writing code in Python 3.9.

Now, before I run `black` for the first time, I like to make sure all my code is committed, so that I can easily see the effect of `black`. I might also run it in `--check` mode first, to make sure it doesn't make any undesirable changes or edit unexpected files:

```
black --verbose --check . tests
```

Sometimes I discover I need to explicitly exclude some Python files from `black`'s formatting rampage. In that case, I add an `exclude` option to `pyproject.toml`:

```
extend-exclude = '''
(
    ^/migrations\
)
'''
```

If all looks good, I'll run `black` for real:

```
black --verbose . tests/
```

Run tests

The built-in testing framework for Python is `unittest`, but another popular framework is `pytest`. That's my new preferred framework.

I configure that by adding another section to `pyproject.toml`:

```
[tool.pytest.ini_options]
addopts = "-ra --cov"
testpaths = ["tests"]
pythonpath = ['.']
```

The `addopts` option adds arguments to the `pytest` command, `--ra` for displaying extra test summary info and `--cov` for running `coverage.py` while running tests. That `--cov` option works only because I `coverage` and `pytest-cov` earlier. The `testpaths` option tells `pytest` where to find the tests, and `pythonpath` helps tests figure out where to import the modules from.

Now I can run all the tests:

```
pytest
```

That will show me which tests passed or failed. If I want to see the coverage report, I can run:

```
coverage report
```

Install pre-commit hook

All of those tools are great, but it's way too easy to forget to run them. That's why pre-commit hooks are handy; they'll run commands before allowing a git commit to go through.

I like to use the `pre-commit framework` to set up hooks as it supports a wide range of commands across multiple languages (and many of my projects are multi-language, like Python + JS).

I create a `.pre-commit-config.yaml` file to describe my desired hooks:

```
repos:
- repo: https://github.com/pre-commit/pre-commit-hooks
  rev: v2.3.0
  hooks:
  - id: check-yaml
  - id: end-of-file-fixer
  - id: trailing-whitespace
- repo: https://github.com/psf/black
  rev: 22.3.0
  hooks:
  - id: black
    args: ['--config=./pyproject.toml']
- repo: https://github.com/charliermarsh/ruff-pre-commit
  rev: v0.0.116
  hooks:
```

```
- id: ruff
```

Those hooks run `black`, `ruff`, as well as some generally useful checks for YAML syntax and trailing whitespace issues.

Now I have to actually install the hooks, which will update my local Git configuration:

```
pre-commit install
```

I can do a test commit now to see if it spots any issues. Most likely I'll have already seen any issues in earlier commits, but it's good to make sure the hooks are run.

You might have noticed that I did *not* run any tests in the pre-commit hooks. That's because tests can often take a number of minutes to run, and for many repositories, running tests on every commit would significantly slow down development and discouraging frequent commits. However, I will run tests using the next tool.

Setup Github action

Using [Github actions](#), I can make sure certain commands are always run upon updates to the Github repo.

To configure an action, I create a `python.yml` file inside a newly created `.github/workflows` folder and describe the desired action following their format:

```
name: Python checks
on: [push, pull_request]

jobs:
  build:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v3
      - name: Set up Python 3
        uses: actions/setup-python@v3
        with:
          python-version: 3.9
      - name: Install dependencies
        run: |
          python -m pip install --upgrade pip
          pip install -r requirements-dev.txt
      - name: Lint with ruff
        run: |
          # stop the build if there are Python syntax errors or undefined names
          ruff . --select=E9,F63,F7,F82
          # exit-zero treats all errors as warnings.
          ruff . --exit-zero
      - name: Check formatting with black
        uses: psf/black@stable
        with:
          src: ". tests/"
          options: "--check --verbose"
      - name: Run unit tests
        run: |
          pytest
```

The `on: [push, pull_request]` tells Github to run the action whenever code is pushed to any branch or whenever pull requests are opened. The `jobs` section describes just one job, "build", which runs on an Ubuntu box with Python 3.9 installed, installs my project requirements, runs `ruff`, runs `black`, and finally, runs `pytest`.

All together now

You can see my sample repository here:

<https://github.com/pamelafox/mypythonproject-sept7th>

You can also make a new repo with a similar setup using [this Github template repository](#). It's similar to the sample repo, but with less actual code, so it's easier to start fresh. It also may not exactly match the tools in this blog post, since I'm always finding ways to improve my workflow.

You can see how I've used this sort of setup in apps which involve other dependencies as well, like:

- [translation-telephone](#): A web app that also uses PostgreSQL, Flask, SQL-Alchemy.
- [recursive-visualizations](#): A web app whose only Python use is in Pyodide (a WASM port of Python).

- [pamelafox-site](#): A web app built with Flask (no DB).
- [django-quiz-app](#): A web app built with Django and PostgreSQL.

Let me know what your project setup is like and if you have suggestions for improving my workflow. I'm always learning! 🤖

Posted by [Pamela Fox](#)

Labels: [python](#)

No comments:

[Post a Comment](#)

[Newer Post](#)

[Home](#)

[Older Post](#)

Subscribe to: [Post Comments \(Atom\)](#)